P3904R0
When paths go WTF: making formatting lossless

Published Proposal, 2025-10-30

Author:
Victor Zverovich

Audience:
SG16

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Table of Contents

"Compatibility means deliberately repeating other people's mistakes." – David Wheeler

§ 1. Introduction

[P2845], adopted in C++26, added formatting support for `std::filesystem::path`, addressing encoding issues and making formatting lossless except for one case, unpaired surrogates on Windows. This paper proposes addressing this case and making formatting 100% lossless by default via the WTF-8 encoding ([WTF-8]). This will improve consistency in path handling between Windows and POSIX platforms and align with the design of `std::format` where the default formatting is normally lossless.

§ 2. Motivation

[P2845] made it possible to format and print Unicode paths, even on Windows, which historically had problems because of legacy code pages. For example

```
std::print("{}\n", std::filesystem::path(L"Шчучыншчына"));
```

is correctly formatted and printed on Windows when the literal encoding is UTF-8 regardles of the Active Code Page.

However, paths are not guaranteed to be valid Unicode or even text. In general they are just sequences of bytes (or 16-bit values on Windows) which often but not always contain Unicode text, quoting [WIN32-FILEIO]:

> the file system treats path and file names as an opaque sequence of `WCHAR`s

This is also true on POSIX ([PEP383]):

> File names, environment variables, and command line arguments are defined as being character data in POSIX; the C APIs however allow passing arbitrary bytes - whether these conform to a certain encoding or not.

Arbitrary paths are formatted on POSIX such that there is no data loss. Unfortunately this is not the case on Windows, for example:

```
auto p1 = std::filesystem::path(L"\xD800"); // a lone surrogate
auto p2 = std::filesystem::path(L"\xD801"); // another lone surrogate
auto s1 = std::format("{}\n", p1); // s1 == "🤷"
auto s2 = std::format("{}\n", p2); // s2 == "🤷"
```

Apart from being inconsistent between platforms, this makes it impossible to reliably round trip paths. For example, `p1` and `p2` above are two distinct paths that are formatted as the same string. This may result in a silent data loss and is remarkably different from other standard formatters such as the ones for floating point numbers which are specifically designed to round trip.

For comparison, on POSIX formatting of arbitrary paths including the ones that are not valid Unicode works as expected and is lossless:

```
auto p = std::filesystem::path("\x80");
auto s = std::format("{}\n", p); // s == "\x80"
```

§ 3. Proposal

The current paper proposes preventing data loss and formatting ill-formed UTF-16 paths using WTF-8 (Wobbly Transformation Format – 8-bit) which is "a superset of UTF-8 that can losslessly represent arbitrary sequences of 16-bit code unit (even if ill-formed in UTF-16) but preserves the other well-formedness constraints of UTF-8." ([WTF-8])

Code	Before	After
`std::format("{}\n", std::filesystem::path(L"\xD800"));`	`"�"`	`"\xED\xA0\x80"`
`std::format("{}\n", std::filesystem::path(L"\xD801"));`	`"�"`	`"\xED\xA0\x81"`

This will enable round trip of paths from `char` strings which is currently not possible. The API for the read path of the round trip will be proposed by a separate paper.

At the same time this will preserve the observable behavior for `std::print` when printing to a terminal. For example:

```
std::print("{}\n", std::filesystem::path(L"\xD800"));
```

will still print

```
�
```

on implementations that follow the recommended practice from [ostream.formatted.print]:

> *Recommended practice*: For `vprint_unicode`, if invoking the native Unicode API requires transcoding, implementations should substitute invalid code units with U+FFFD REPLACEMENT CHARACTER per the Unicode Standard, Chapter 3.9 U+FFFD Substitution in Conversion.

WTF-8 is used to handle invalid UTF-16 in paths and other system APIs in Rust ([RUST-OSSTRING]) and Node.js libuv ([LIBUV]). Python also handles this but with a different mechanism ([PEP383]).

§ References

§ Informative References

[LIBUV]
libuv contributors. *Miscellaneous utilities. libuv Documentation.*. URL: https://docs.libuv.org/en/v1.x/misc.html

[P2845]

Victor Zverovich. *Formatting of std::filesystem::path*. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p2845r8.html

[PEP383]

Martin von Löwis. *PEP 383 – Non-decodable Bytes in System Character Interfaces*. URL: https://peps.python.org/pep-0383/

[RUST-OSSTRING]

Rust Project Developers. *OsString Struct. The Rust Standard Library.*. URL: https://doc.rust-lang.org/std/ffi/struct.OsString.html

[WIN32-FILEIO]

Microsoft Corporation. *Maximum Path Length Limitation – Local file systems*. URL: https://learn.microsoft.com/en-us/windows/win32/fileio/maximum-file-path-limitation

[WTF-8]

Simon Sapin. *The WTF-8 encoding*. URL: https://wtf-8.codeberg.page/